

processor working at high clock speeds. Now, a processor with a lot of electricity all around the die is prone to electrical noise. This electrical noise causes interference between the 'wires', or pathways, on the processor. There could then be some electrical radiation between adjacent pathways, which could, in turn, corrupt data.

Another solution lies in keeping the processor busy for more of the time. This is achieved via HyperThreading, for example (see box *HyperThreading*).

One more possible solution is increasing the L1 and L2 caches (See box *Processor Caches* for more). This is a definite possibility, and is being done—but cache memory is expensive, and increases the total cost of the chip. Both Intel and AMD have been increasing the size of the L2 caches. Intel has even added an L3 cache to the 'Extreme' Pentium versions—this was previously unheard of in a desktop processor.

However, it must be borne in mind that increasing the cache is a diminishing-returns situation: beyond a certain point, increasing the cache just doesn't help.

As for increasing transistor count, processors are already cramped. While it may be possible to cram more in the future, it doesn't seem feasible right now.

The most elegant solution comes about when you think of the way computers are used these days: one person running several applications at the same time. A quick-fix solution would seem to be, why not delegate two processors for the same jobs? In other words, if there are four tasks being performed, how about two processors for the job—each of them doing two tasks, both working at the same time?

In principle, that's what dual-core processors are about: they're essentially two processor cores on one die. They're not two physically separate



Processor Caches

Processor cache memory is expensive, very fast memory—much more expensive than, say, DDR-RAM. It is located on the same die as the processor. The processor cache is used by the processor to reduce the average time that it takes to access memory.

To understand how cache memory works, think about two instructions—"add A and B to make C" and "add D to C". Now, after the processor has added A and B to make C, C is stored in main memory (the RAM) as a value. But it's also stored in the cache. This is because it's very likely that there will be some instructions in the near future that will need C. Sure enough, our next instruction goes "Add D to C", and the processor, when it looks for the value of C, looks in the cache first. In our example, it will definitely find it in the cache, because just the previous instruction mentioned C. So overall, the CPU has to 'go' to main memory less often, with the help of caches. (Remember that main memory is much slower to respond than cache memory. Since the cache needs to be fast, it has to be placed physically very close to the processor.)

Now where 'L1' and 'L2' come in is when you consider the fact that L1 cache—the memory that is physically closest to the processor, after the actual CPU registers—is prohibitively expensive. Processors therefore can only afford a small L1 cache. If the cache size is to be increased, one needs to resort to a slower, larger L2 cache, which is further away from the processor. This is the multi-level cache concept.

The L1 cache, typically 8 to 64 KB in size, is where the processor looks first. If the item is not found, the processor looks in the larger (typically 256 KB to 2 MB), slower L2 cache, and so on, all the way to main memory.

processors—the cores are on the same die, and internally share the L1 and L2 caches (See box *Processor Caches*). The two cores also have separate pipelines (See box *Instructions And Pipelines*), so data gets processed that much faster. (A die is an integrated circuit chip as diced or cut from a finished wafer; a wafer itself is a thin slice of semi-conducting material, such as a silicon crystal, upon which the microcircuits are constructed.)

The two cores share some hardware, such as the memory controller and the bus, whereas a dual-processor system has completely separate resources—such as the caches—for each processor.

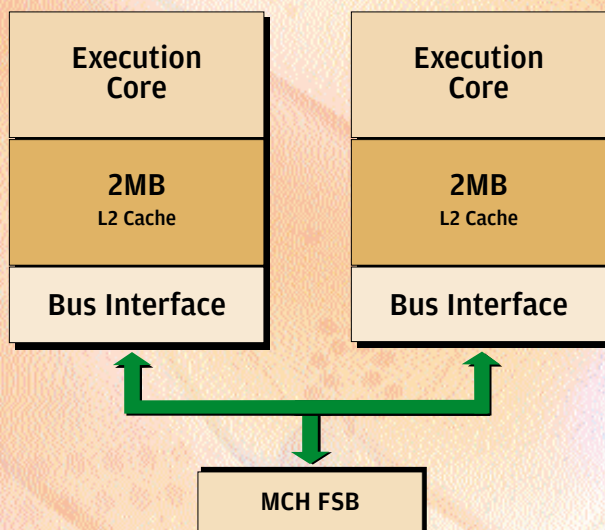
Remember that in general, a dual-core processor won't be as fast as a dual-processor configuration. There could be exceptions to this, depending on the software that runs on these newer systems—so we'll have to wait and watch to see if dual-core beats dual-processor.

So then, why two cores on one die, when you could have a dual-processor rig? The answer is cost. A dual-core processor works out cheaper in terms of production cost than two processors taken together. This is not to mention that the motherboards required for a dual-processor configuration are more expensive too, because of the extra circuitry, and for other reasons as well.

Think now about how two cores would help: the simplest scenario is when your computer is doing two entirely different things. One could be the encoding of a video file, and the other could be the manipulation of an image in Photoshop. These two tasks could, in principle, be handled by the two cores independently. There's another scenario here: the second processor will be able to keep the system running smoothly if the first core is being put to 100 per cent use. Think about how your system slows down when you're encoding a video file... with the help of the second processor



Intel's Dual-Core Approach



The Intel approach to dual-core seems more like a quick-fix solution than something built from the ground up. The two cores have their own L2 caches and bus interfaces, but the communication between the cores happens via the Northbridge and FSB outside the chip. The 'MCH' here is the Memory Controller Hub.